PRACTICAL - 10

Part A — Practice Programs (Hour 1)

PROBLEM1	1. Counter race: several threads each increment a shared counter many times. Run it without synchronization to watch the total come out too low, then make the increment synchronized and confirm it is exactly right every run.
CODE	

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class ThreadPoolRunner {

   public static void main(String[] args) {

      // Create a fixed thread pool with 3 threads
      ExecutorService executor = Executors.newFixedThreadPool(3);

      // Submit 10 tasks
      for (int i = 1; i <= 10; i++) {
         int taskId = i;

         executor.submit(() -> {
            try {
               System.out.println(
                  "Task " + taskId + " is running on " +
                  Thread.currentThread().getName()
               );

               // Sleep briefly
               Thread.sleep(1000);

               System.out.println(
                  "Task " + taskId + " completed by " +
                  Thread.currentThread().getName()
               );

            } catch (InterruptedException e) {
               Thread.currentThread().interrupt();
            }
         });
      }

      // No more tasks will be submitted
      executor.shutdown();
```

```
        try {
            // Wait until all tasks are completed
            if (executor.awaitTermination(30, TimeUnit.SECONDS)) {
                System.out.println("All tasks completed.");
            } else {
                System.out.println("Timeout! Some tasks are still running.");
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 10> javac .\ThreadPoolRunner.java
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 10> java .\ThreadPoolRunner.java
Task 1 is running on pool-1-thread-1
Task 3 is running on pool-1-thread-3
Task 2 is running on pool-1-thread-2
Task 1 completed by pool-1-thread-1
Task 3 completed by pool-1-thread-3
Task 2 completed by pool-1-thread-2
Task 4 is running on pool-1-thread-1
Task 5 is running on pool-1-thread-2
Task 6 is running on pool-1-thread-3
Task 6 completed by pool-1-thread-3
Task 7 is running on pool-1-thread-3
Task 5 completed by pool-1-thread-2
Task 4 completed by pool-1-thread-1
Task 8 is running on pool-1-thread-2
Task 9 is running on pool-1-thread-1
Task 7 completed by pool-1-thread-3
Task 10 is running on pool-1-thread-3
Task 8 completed by pool-1-thread-2
Task 9 completed by pool-1-thread-1
Task 10 completed by pool-1-thread-3
All tasks completed.
```

PROBLEM2

2. Producer–consumer: one thread adds numbers to a small shared buffer and another removes them, coordinated with wait()/notify(); produce and consume a fixed number of items in order with none lost.

CODE

```
class SharedBuffer {

    private int item;
    private boolean available = false;

    // Producer adds an item
    public synchronized void produce(int value) throws InterruptedException {

        // Wait if buffer is full
        while (available) {
            wait();
        }
```

```
        item = value;
        available = true;

        System.out.println("Produced: " + value);

        // Notify consumer
        notify();
    }

    // Consumer removes an item
    public synchronized int consume() throws InterruptedException {

        // Wait if buffer is empty
        while (!available) {
            wait();
        }

        int value = item;
        available = false;

        System.out.println("Consumed: " + value);

        // Notify producer
        notify();

        return value;
    }
}

public class ProducerConsumer {

    public static void main(String[] args) {

        SharedBuffer buffer = new SharedBuffer();

        int numberOfItems = 10;

        // Producer thread
        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= numberOfItems; i++) {
                    buffer.produce(i);
                    Thread.sleep(200);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
```

```
        });

        // Consumer thread
        Thread consumer = new Thread(() -> {
            try {
                for (int i = 1; i <= numberOfItems; i++) {
                    buffer.consume();
                    Thread.sleep(300);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        producer.start();
        consumer.start();

        try {
            producer.join();
            consumer.join();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        System.out.println("All items produced and consumed.");
    }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 10> javac .\ProducerConsumer.java
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 10> java .\ProducerConsumer.java
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
Produced: 6
Consumed: 6
Produced: 7
Consumed: 7
Produced: 8
Consumed: 8
Produced: 9
Consumed: 9
Produced: 10
Consumed: 10
All items produced and consumed.
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 10>
```

PROBLEM3	3. (For Advanced Learners – home) Reproduce a deadlock between two threads that lock two objects in opposite order, then remove it with a consistent lock order. Choose the scenario yourself.

CODE	public class DeadlockDemo {
	
	static Object accountA = new Object();
	static Object accountB = new Object();
	
	// This method can cause DEADLOCK
	static void transferAtoB() {
	
	synchronized (accountA) {
	System.out.println("Thread 1 locked Account A");
	
	try {
	Thread.sleep(100);
	} catch (InterruptedException e) {
	Thread.currentThread().interrupt();
	}
	
	synchronized (accountB) {
	System.out.println("Thread 1 locked Account B");
	}
	}
	}
	
	// Locks objects in the OPPOSITE order
	static void transferBtoA() {
	
	synchronized (accountB) {
	System.out.println("Thread 2 locked Account B");
	
	try {
	Thread.sleep(100);
	} catch (InterruptedException e) {
	Thread.currentThread().interrupt();
	}
	
	synchronized (accountA) {
	System.out.println("Thread 2 locked Account A");
	}
	}
	}
	
	public static void main(String[] args) {

```
        Thread thread1 = new Thread(DeadlockDemo::transferAtoB);
        Thread thread2 = new Thread(DeadlockDemo::transferBtoA);

        thread1.start();
        thread2.start();
    }
}
```

OUTPUT

```
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 10> javac .\DeadlockDemo.java
PS C:\Users\aimlo\OneDrive\Desktop\CEUE203 OOP\Practical 10> java .\DeadlockDemo.java
Thread 1 locked Account A
Thread 2 locked Account B
```